
PERPETUAL MOTION SQUAD
POCKETEXPENSE+

Test Case Documentation

Production-Grade Offline-First Financial Tracking System

Purvansh Sahu Yash Arya Aman Anand

TEAM	Perpetual Motion Squad
DEGREE	BSc (Hons) in Computer Science
INSTITUTION	BITS Pilani
PROJECT GUIDE	Mr. Murali Raghu
DATE	August 2026
DOCUMENT	Test Case Documentation

1. Test Case Summary

Total: 198 automated tests (96 backend + 102 frontend) · **Backend coverage:** ~82% (81.81% measured) · **SMS accuracy:** 99% on a 200-sample corpus · 0% false positives · 0.92 average confidence

Status: All 198 tests passing · 0 failures · 0 open defects.

Area	Suite	Tests	Result
Backend (96)	Expense Service	15	PASS
	Budget Service	12	PASS
	Recurring Service	10	PASS
	Insight Service	11	PASS
	API Integration	17	PASS
	Edge Cases	31	PASS
Frontend (102)	SMS Parser	42	PASS
	Receipt Parser	16	PASS
	Export Format	16	PASS
	Budget Thresholds	12	PASS
	Bulk Delete	4	PASS
	Stats	12	PASS
Total		198	198 PASS · 0 FAIL

Conventions. Test cases are identified as **TC-Bxxx** (backend) and **TC-Fxxx** (frontend). Priority: **High** = financial correctness / security; **Medium** = core functionality; **Low** = polish. Each row records the verified outcome; all recorded results are **PASS**.

2. Backend Test Cases (96)

2.1 Expense Service (TC-B01–B15) • coverage 91%

ID	Scenario	Pri	Result
TC-B01	Create valid expense persists all required fields	High	PASS
TC-B02	Defaults applied (date, syncStatus=synced)	Med	PASS
TC-B03	Rejects invalid category	High	PASS
TC-B04	Rejects invalid paymentMethod	High	PASS
TC-B05	Rejects negative or zero amount	High	PASS
TC-B06	Recurring expense computes nextRunDate by frequency	High	PASS
TC-B07	List returns only the requesting user's expenses	High	PASS
TC-B08	Pagination (page, limit) applied correctly	Med	PASS
TC-B09	Returns totalItems / totalPages metadata	Med	PASS
TC-B10	Update modifies fields and recomputes recurring dates	High	PASS
TC-B11	Update rejects invalid payloads	High	PASS
TC-B12	Delete removes the record	High	PASS
TC-B13	Sync maps localId → serverId for new records	High	PASS
TC-B14	Sync skips already-synchronised records	Med	PASS
TC-B15	Sync applies tombstone deletions	High	PASS

2.2 Budget Service (TC-B16–B27) • coverage 93%

ID	Scenario	Pri	Result
TC-B16	Create valid monthly category budget	High	PASS
TC-B17	Rejects duplicate user+category+month+year	High	PASS
TC-B18	Rejects amount below zero	High	PASS
TC-B19	Rejects invalid month (outside 1–12)	High	PASS
TC-B20	List returns only current-month budgets for the user	Med	PASS
TC-B21	Update amount recalculates virtual fields	High	PASS
TC-B22	Delete removes the record	Med	PASS
TC-B23	$\text{percentageUsed} = (\text{totalSpent} / \text{amount}) \times 100$	High	PASS
TC-B24	$\text{remainingAmount} = \max(\text{amount} - \text{totalSpent}, 0)$	High	PASS
TC-B25	Recalculate totalSpent aggregates matching expenses	High	PASS
TC-B26	Expense create/update/delete triggers recalculation	High	PASS
TC-B27	Recalculation resets totalSpent to 0 with no expenses	Med	PASS

2.3 Recurring Service (TC-B28–B37) • coverage 91%

ID	Scenario	Pri	Result
TC-B28	Daily recurrence advances nextRunDate by 1 day	High	PASS
TC-B29	Weekly recurrence advances by 7 days	High	PASS

TC-B30	Monthly recurrence advances by 1 month	High	PASS
TC-B31	Processor clones each due record	High	PASS
TC-B32	Processor sets lastProcessedDate	High	PASS
TC-B33	Idempotency: re-run does not duplicate	High	PASS
TC-B34	Skips future nextRunDate records	Med	PASS
TC-B35	Triggers budget recalculation	Med	PASS
TC-B36	Non-recurring expenses never processed	Med	PASS
TC-B37	Concurrent cron invocations remain idempotent	High	PASS

2.4 Insight Service (TC-B38–B48) • coverage 91%

ID	Scenario	Pri	Result
TC-B38	Monthly growth returns rates over 6-month window	Med	PASS
TC-B39	Month-over-month % change computed correctly	High	PASS
TC-B40	Handles month with zero prior spending	Med	PASS
TC-B41	Top 3 categories with count and average	High	PASS
TC-B42	Weekday vs weekend totals / counts / averages	High	PASS
TC-B43	Comparison string computed	Low	PASS
TC-B44	Flags expenses > 2 std deviations	High	PASS
TC-B45	Returns z-scores with items	High	PASS
TC-B46	No anomalies for uniform data	Med	PASS
TC-B47	Empty results for user with no expenses	Med	PASS
TC-B48	Scopes strictly to requesting user's data	High	PASS

2.5 API Integration (TC-B49–B65) • 100% route coverage

ID	Scenario	Pri	Result
TC-B49	Register creates user and returns token	High	PASS
TC-B50	Register rejects duplicate email	High	PASS
TC-B51	Register rejects invalid email format	High	PASS
TC-B52	Login succeeds with valid credentials	High	PASS
TC-B53	Login rejects wrong password	High	PASS
TC-B54	GET profile requires valid JWT	High	PASS
TC-B55	PUT profile updates name/budgetLimit/currency	Med	PASS
TC-B56	GET /api/expenses requires auth	High	PASS
TC-B57	POST /api/expenses validated via Joi	High	PASS
TC-B58	GET /api/expenses/:id returns the expense	Med	PASS
TC-B59	PUT /api/expenses/:id updates	High	PASS
TC-B60	DELETE /api/expenses/:id removes	High	PASS
TC-B61	POST /api/expenses/sync reconciles offline data	High	PASS
TC-B62	GET /api/budgets returns current-month budgets	Med	PASS
TC-B63	POST /api/budgets creates (validated)	High	PASS
TC-B64	GET /api/insights/advanced returns payload	Med	PASS

TC-B65	GET /api/health returns success:true	Low	PASS
--------	--------------------------------------	-----	------

2.6 Edge Cases (TC-B66–B96)

ID	Category	Scenario	Pri	Result
TC-B66	Zero data	List with no records returns empty page	Med	PASS
TC-B67	Zero data	Budgets with no expenses report 0%	Med	PASS
TC-B68	Zero data	Insights with no data return empty aggregates	Med	PASS
TC-B69	Volume	Pagination handles 1000+ records	Med	PASS
TC-B70	Volume	Limit capped at 100	Med	PASS
TC-B71	Volume	Page beyond last returns empty list	Low	PASS
TC-B72	Concurrency	Simultaneous creates do not corrupt totals	High	PASS
TC-B73	Concurrency	Concurrent sync requests stay idempotent	High	PASS
TC-B74	Concurrency	Duplicate budget creation rejected (unique index)	High	PASS
TC-B75	Precision	Decimal amounts keep precision (1250.50)	High	PASS
TC-B76	Precision	No rounding drift on aggregation	High	PASS
TC-B77	Precision	Large amount at ₹10,00,000 guardrail accepted	Med	PASS
TC-B78	Timezone	Date filtering respects day boundaries	High	PASS
TC-B79	Timezone	Month aggregation scopes correctly	High	PASS
TC-B80	Validation	Unknown fields stripped from requests	Med	PASS
TC-B81	Validation	Cast errors on invalid ObjectId handled	High	PASS
TC-B82	Validation	Duplicate-key errors structured	Med	PASS
TC-B83	Auth	Expired token rejected	High	PASS
TC-B84	Auth	Malformed token rejected	High	PASS
TC-B85	Auth	Cross-user expense access denied	High	PASS
TC-B86	Auth	Auth rate limiter blocks brute force (20/15min)	High	PASS
TC-B87	Auth	API rate limiter blocks excess (100/15min)	High	PASS
TC-B88	Recurring	Month-end recurring stays correct	High	PASS
TC-B89	Recurring	31st monthly recurrence rolls over	High	PASS
TC-B90	Sync	Offline pending update reconciled	High	PASS
TC-B91	Sync	Tombstone delete prevents resurrection	High	PASS
TC-B92	Sync	Empty sync returns empty result	Low	PASS
TC-B93	Budget	totalSpent respects category/month scope	High	PASS
TC-B94	Budget	Deleting an expense reduces totalSpent	High	PASS
TC-B95	Budget	Income does not count toward spend	High	PASS
TC-B96	Robustness	Health check responds under load	Med	PASS

3. Frontend Test Cases (102)

3.1 SMS Parser (TC-F01–F42) • 200-sample corpus (120 debit, 40 credit, 20 OTP, 20 promo)

ID	Scenario	Pri	Result
TC-F01-03	normalizeMessage: strips zero-width chars, normalises ₹/INR, preserves decimals	High	PASS
TC-F04-09	Sender detection: HDFCBK, SBIINB, ICICIB, AXISBK, KOTAKB, generic BANK	High	PASS
TC-F10-13	Classification: debit / credit / OTP ignore / promo ignore	High	PASS
TC-F14-16	Strict decimal parsing: "Rs.450.00", "INR 3,450.00", rejects malformed	High	PASS
TC-F17-19	Field extraction: merchant, account last-4, timestamp	High	PASS
TC-F20-23	Guardrails: ₹1 min, ₹10,00,000 max, merchant ≥2 chars, min SMS length 15	High	PASS
TC-F24-26	Confidence +0.25 sender, +0.30 amount, +0.20 merchant	High	PASS
TC-F27-29	Penalties -0.20 ambiguous, -0.15 multi-amount, -0.10 suspicious format	High	PASS
TC-F30	Sender gating: unrecognised sender <0.65 ignored	High	PASS
TC-F31-33	Routing: ≥0.75 high-confidence, 0.50-0.74 low, <0.50 ignored	High	PASS
TC-F34-35	Deduplication: repeat rejected; hash = amount+merchant+type+acct+date	High	PASS
TC-F36-38	Category auto-detection: Swiggy→food, Amazon→shopping, Uber→travel	Med	PASS
TC-F39	Full 200-sample accuracy ≥ 95%	High	PASS (99%)
TC-F40	Full 200-sample false positives ≤ 3%	High	PASS (0%)
TC-F41	All 20 OTP samples rejected (100%)	High	PASS
TC-F42	All 20 promo samples rejected (100%)	High	PASS

3.2 Receipt Parser (TC-F43–F58)

ID	Scenario	Pri	Result
TC-F43-46	Amount by weighted labels: grand total / total payable / total / amount-paid	High	PASS
TC-F47-49	Rejects subtotal / tax / discount / GSTIN / identifier lines	High	PASS
TC-F50-52	Merchant (stopword-filtered); date from locale/numeric formats	Med	PASS
TC-F53-54	Confidence 0-1; candidate amounts largest-first	Med	PASS
TC-F55-56	₹1-₹10,00,000 guardrail; null over wrong amount	High	PASS
TC-F57-58	Malformed / empty receipt handled gracefully	Med	PASS

3.3 Export Format (TC-F59–F74)

ID	Scenario	Pri	Result
TC-F59-62	csvCell escapes commas, quotes, newlines; doubles embedded quotes (RFC 4180)	High	PASS
TC-F63-66	toCSV signs expenses - / income +; headers; labels; trailing newline	High	PASS
TC-F67-68	summarise groups by category; empty set handled	Med	PASS
TC-F69-73	selectForExport windows: this month / last month / 3 months / all; oldest-first	High	PASS
TC-F74	escapeHtml neutralises markup for PDF safety	High	PASS

3.4 Budget Thresholds (TC-F75–F86) • findCrossings

ID	Scenario	Pri	Result
TC-F75-76	Quiet below threshold; warns at ≥ threshold (80%)	High	PASS
TC-F77-78	Exceeded at 100%; never both warning + exceeded	High	PASS
TC-F79-80	Once-per-period dedup; different scope still warns	High	PASS

TC-F81-82	Honours notifyOnExceed off; ignores no-limit scopes	Med	PASS
TC-F83-85	Custom threshold; remaining-amount in copy; scope label	High	PASS
TC-F86	Every scope evaluated independently	High	PASS

3.5 Bulk Delete (TC-F87–F90) • deleteExpenses reducer

ID	Scenario	Pri	Result
TC-F87-88	Removes matching items from items and pendingQueue	High	PASS
TC-F89	Empty selection is a no-op	Med	PASS
TC-F90	Offline deletes tracked via tombstones	High	PASS

3.6 Stats (TC-F91–F102)

ID	Scenario	Pri	Result
TC-F91-95	totalsForMonth: empty month, month-only scope, boundaries, expense/income split, year isolation	High	PASS
TC-F96-97	monthOverMonthChange: increase +, decrease –	High	PASS
TC-F98-100	Zero base, both zero, divide-by-zero avoided	High	PASS
TC-F101-102	Matches home badge logic and displayed balance	High	PASS

4. Quality Metrics

SMS PARSER ACCURACY REPORT

```

Total samples:      200    Accuracy:      99%
True positives:     158    False + rate: 0%
True negatives:      40    Avg confidence: 0.92
False positives:      0
False negatives:      2

```

Metric	Target	Actual	Result
Detection accuracy	≥ 95%	99%	PASS
False positive rate	≤ 3%	0%	PASS
Debit detection rate	≥ 95%	100%	PASS
Credit detection rate	≥ 90%	95%	PASS
OTP rejection	100%	100%	PASS
Promo rejection	100%	100%	PASS
Avg confidence score	≥ 0.6	0.92	PASS
Backend statement coverage	—	~82%	PASS
Backend route coverage	100%	100%	PASS

5. Execution Commands

5.1 Backend (96)

```
cd server
npm test                # all 96
npm run test:unit        # 48 service-level
npm run test:integration # 17 API-level
npm run test:edge        # 31 edge cases
npm run test:coverage    # ~82% coverage
```

Expected: **Test Suites: 6 passed · Tests: 96 passed, 96 total.** Backend tests use mongodb-memory-server; no database setup required.

5.2 Frontend (102)

```
npm test                # all 102
npm run test:sms        # SMS parser suite only (42)
```

Expected: **Test Suites: 6 passed · Tests: 102 passed, 102 total.** The SMS suite prints the accuracy report in §4.

5.3 Static checks & full verification

```
npx tsc --noEmit        # 0 errors
npx expo lint           # 0 errors
npx tsc --noEmit        && \
npx expo lint           && \
npm test                && \
npm test --prefix server
```

5.4 Load testing

```
cd server
npm install -g artillery
npm start                # terminal 1
artillery run tests/performance/artillery.yml # terminal 2
```

Targets enforced by configuration: p95 < 500 ms · p99 < 1000 ms · 100 concurrent users at 500 req/min.

Conclusion: All 198 test cases pass across six backend and six frontend suites — zero failures, zero open defects.